57

BÖLÜM: GITHUB ACTIONS CI/CD & BÜYÜK MEZUNIYET SENARYOSU | MODULE: GITHUB ACTIONS CI/CD & THE GRAND ENTERPRISE FINALE

Docker İmajı Paketleme, Trivy Güvenlik Taraması ve GHCR Push

Docker Packaging, Trivy Vulnerability Scanning & GHCR Push

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Fabrikadan çıkan Spring Boot uygulamasını çelik bir kargo konteynerine koyuyoruz. Ama hemen gemiye yüklemiyoruz! Önce konteyneri X-Ray cihazından (Trivy güvenlik tarayıcısı) geçiriyoruz. İçinde tehlikeli bir virüs veya güvenlik açığı (CVE) var mı? Temiz raporu çıkarsa, GitHub Container Registry'ye (GHCR) resmi şirket mührüyle teslim ediyoruz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

We pack our freshly compiled Spring Boot binary into a standard steel container. But we don't dispatch it blindly! First, it passes through a high-tech airport X-Ray scanner (Trivy vulnerability scanner). Are there any high-severity CVEs? Only when it receives a clean bill of health do we push it with an immutable cryptographic tag to GHCR!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Bu aşama üç kritik adımdan oluşur: 1) `docker/setup-buildx-action`: Docker Buildx ile hızlı, çok katmanlı önbellek kullanan derleme altyapısı kurar. 2) `aquasecurity/trivy-action`: Üretilen konteyner imajını işletim sistemi paketleri ve Java JAR bağımlılıklarındaki (Log4Shell vb.) bilinen CVE açıkları için tarar; `severity: 'CRITICAL'` bulunursa pipeline'ı keser (`exit-code: 1`). 3) `ghcr.io`: GitHub'ın yerleşik OCI uyumlu konteyner deposuna güvenli push yapılır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

This stage executes: 1) Docker Buildx initialization for advanced multi-platform and layer caching. 2) Trivy vulnerability analysis scanning both OS packages and application dependencies against national vulnerability databases; fails the build on Critical CVE thresholds. 3) Authenticated pushing to GitHub Container Registry (`ghcr.io`) using short-lived runner permissions.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# Docker Derleme, Tarama ve GHCR Push:
- name: GitHub Container Registry Girişi
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- name: Docker İmajını Derle
  uses: docker/build-push-action@v5
  with:
    context: .
    load: true
    tags: ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}

- name: Trivy ile Güvenlik Taraması Yap
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: 'ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}'
    severity: 'CRITICAL,HIGH'
    exit-code: '0' # Kritik açıkları raporla

- name: İmajı Registry'ye Gönder
  run: docker push ghcr.io/${{ github.repository }}/coffee-app:${{ github.sha }}
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

İmaj etiketi olarak ASLA `:latest` kullanmayın! `:latest` etiketi hangi kodun canlıda çalıştığını anlamayı imkansız kılar ve K8s imaj önbelleği yüzünden güncellemelerin çekilmemesine neden olur. Her zaman git commit SHA'sını (`: ${{ github.sha }}`) kullanın.

🇬🇧 WATCH OUT

NEVER tag images with `:latest` in production pipelines! `:latest` obscures version provenance and defeats Kubernetes image caching. Always tag with the immutable Git commit SHA.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Trivy güvenlik açığı olan paketlerin üretim ortamına sızmasını önler.
- GHCR doğrudan GitHub ekosistemiyle entegre, yüksek hızlı imaj barındırır.
- Her imaj benzersiz Git SHA etiketiyle mühürlenerek tam izlenebilirlik sağlanır.

- Automated Trivy scanning blocks known vulnerable dependencies from reaching production.
- GHCR offers seamless authentication and high-bandwidth image distribution.
- Tagging with Git commit SHA guarantees 100% traceability between code and containers.